

Relatório Executivo

Avaliação de Métricas em Modelos de Classificação Detecção de Falhas em Sensores Industriais — Indústria 4.0

Unidade Curricular: Engenharia de Dados e MLOps | Prof. MSc. Hugo Menezes Barra | Gerado em 03/09/2026 09:14

1. Arquitetura da Solução e Prevenção de Data Leakage

O pipeline foi construído em módulos independentes (**data_generation** → **preprocessing** → **modeling** → **metrics** → **threshold_analysis** → **report**), partindo de um dataset sintético reprodutível de 20.000 leituras de seis sensores (vibração, temperatura, corrente, pressão, rotação e ruído), com 0.5% de eventos de falha. A semente aleatória fixa garante que qualquer execução reproduza exatamente os mesmos resultados.

1.1 Como o isolamento preveniu o Data Leakage

Data Leakage ocorre quando informação do conjunto de teste influencia, direta ou indiretamente, o treinamento. O erro clássico é aplicar o *StandardScaler* no dataset completo antes do split: média e desvio-padrão passariam a carregar estatística das amostras de teste, e a métrica final ficaria otimista demais — um modelo que parece ótimo no notebook e falha na planta.

Neste projeto a ordem foi rigorosamente invertida: **(1)** o *train_test_split* estratificado é a primeira operação sobre os dados; **(2)** o *StandardScaler* executa **fit_transform** apenas no treino; **(3)** no teste é chamado somente **transform**, reaproveitando os parâmetros aprendidos. A estratificação é indispensável: com apenas 0,5% de positivos, um split puramente aleatório poderia concentrar quase todas as falhas de um lado só.

Conjunto	Amostras	Falhas	% de falha
Treino	14.000	70	0.500%
Teste	6.000	30	0.500%

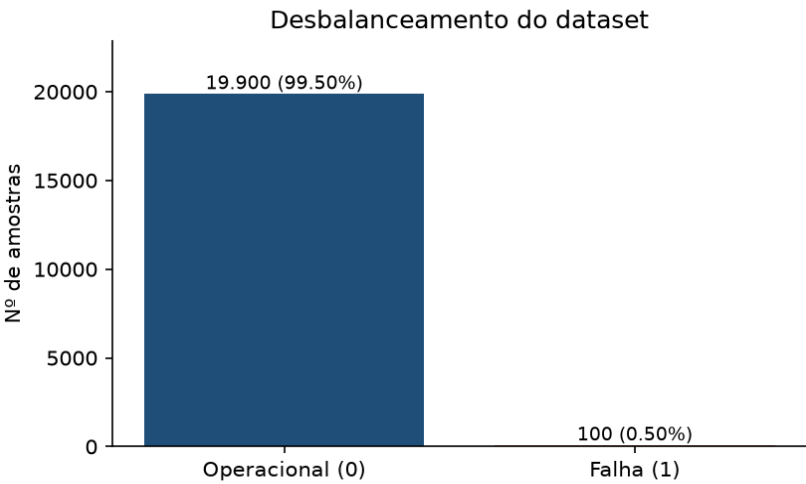


Figura 1 — Distribuição das classes no dataset gerado.

1.2 Modelo

Foi utilizada uma **Regressão Logística** com **class_weight='balanced'**. O parâmetro reponderiza a função de custo pelo inverso da frequência de cada classe, fazendo um erro sobre uma falha pesar cerca de 200× mais do que um erro sobre uma leitura operacional. Sem esse ajuste, o otimizador convergiria para a solução trivial de nunca prever falha.

2. Quadro Comparativo de Métricas

A tabela compara três configurações sobre o **mesmo conjunto de teste isolado**: o baseline ingênuo (prevê sempre 'Operacional'), o modelo no threshold padrão de 0,50 e o modelo no threshold otimizado por custo.

Métrica	Baseline ingênuo	Modelo (t = 0,50)	Modelo (t = 0.33)
Acurácia	0.9950	0.9798	0.9728
Verdadeiros Positivos (TP)	0	28	30
Verdadeiros Negativos (TN)	5.970	5.851	5.807
Falsos Positivos (FP)	0	119	163
Falsos Negativos (FN)	30	2	0
Precisão	0.0000	0.1905	0.1554
Recall (Sensibilidade)	0.0000	0.9333	1.0000
Especificidade	1.0000	0.9801	0.9727
F ₁ -Score	0.0000	0.3164	0.2691
F ₂ -Score	0.0000	0.5243	0.4792
F _{0.5} -Score	0.0000	0.2265	0.1870
AUC-ROC	0.5000	0.9971	0.9971

2.1 A armadilha da Acurácia

O baseline ingênuo atinge **99.50% de acurácia** sem detectar uma única falha (TP = 0, Recall = 0). Ele simplesmente reproduz a frequência da classe majoritária. Na prática, esse modelo deixaria passar as 30 falhas do conjunto de teste, o que representaria **R\$ 1.500.000,00** em paradas não planejadas. Isso demonstra por que a acurácia é uma métrica inútil — e perigosa — em cenários com 0,5% de positivos.

2.2 Interpretação da família F-beta

A família $F_\beta = (1 + \beta^2) \times (P \times R) / (\beta^2 \times P + R)$ permite escolher o peso relativo entre precisão e recall. **F₁** trata as duas igualmente; **F₂** dá peso 4× maior ao recall e é a métrica adequada para manutenção preditiva, onde deixar de detectar uma falha (FN) custa muito mais caro que um alarme falso; **F_{0.5}** privilegia a precisão e seria preferível apenas se cada inspeção fosse extremamente cara ou disruptiva.

A **AUC-ROC de 0.9971** mede a capacidade de ordenação do modelo independentemente do corte escolhido — por isso é idêntica nos dois thresholds. Ela indica a probabilidade de o modelo atribuir score maior a uma falha real do que a uma leitura operacional escolhida ao acaso.

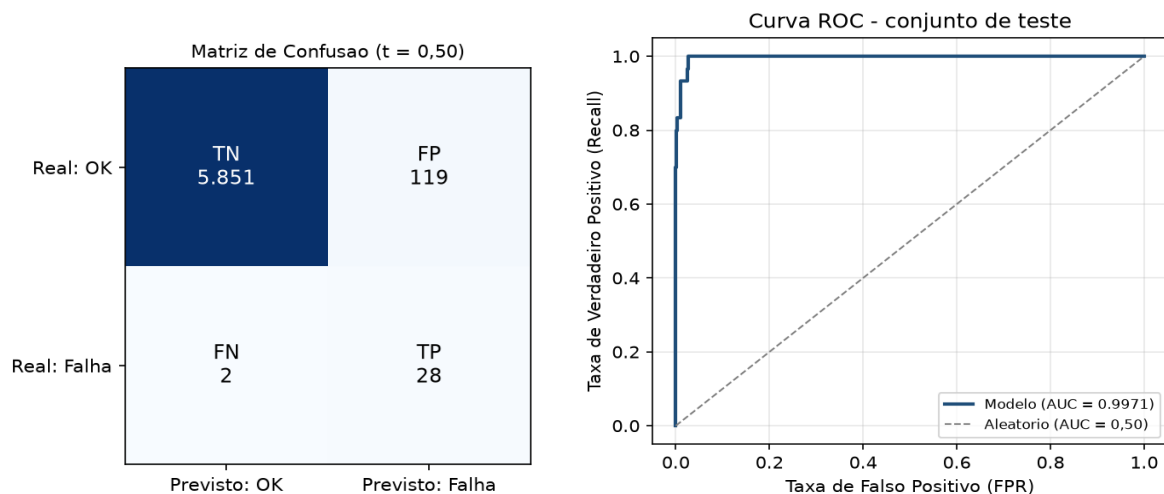


Figura 2 — Matriz de confusão (t = 0,50) e Curva ROC no conjunto de teste.

3. Análise Financeira do Threshold

O threshold 0,50 é uma convenção estatística, não uma decisão de negócio. Numa planta fabril os dois erros têm custos assimétricos: um **Falso Negativo** (falha não detectada) gera parada não planejada de linha, estimada em **R\$ 50.000,00**; um **Falso Positivo** (alarme falso) gera apenas uma inspeção preventiva desnecessária, estimada em **R\$ 800,00** — uma razão de 62:1.

A rotina de ajuste dinâmico varre 99 thresholds entre 0,01 e 0,99 e minimiza a função de custo: **Custo Total(t) = FN(t) × custo_FN + FP(t) × custo_FP**.

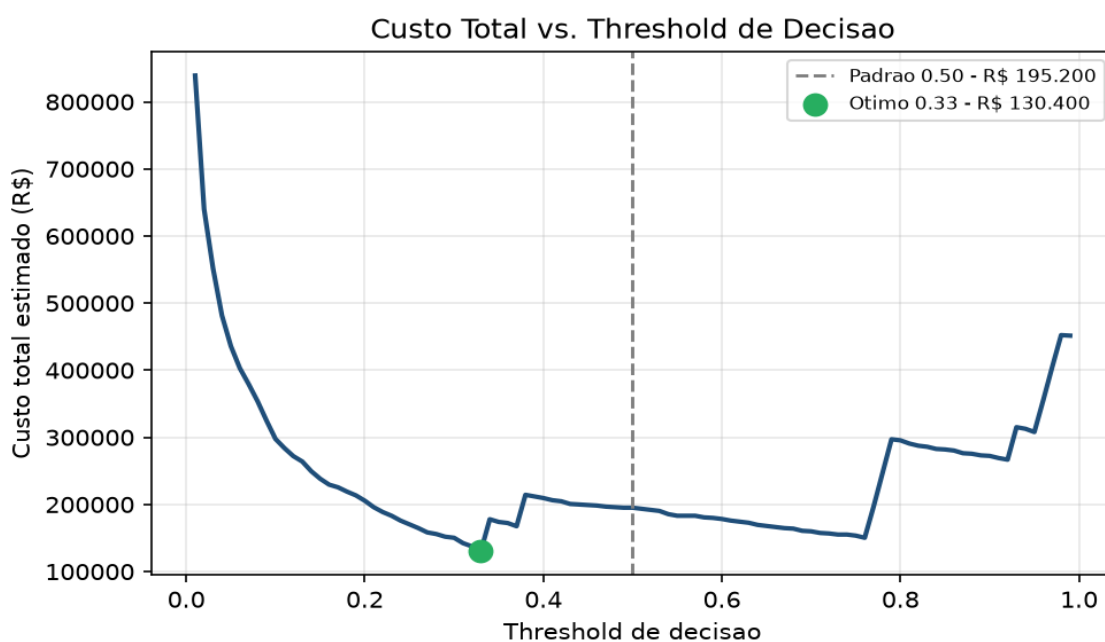


Figura 3 — Curva de Custo Total vs. Threshold, com o ponto de menor custo destacado.

Cenário	Threshold	FN	FP	Recall	Custo Total
Padrão	0.50	2	119	0.9333	R\$ 195.200,00
Otimizado	0.33	0	163	1.0000	R\$ 130.400,00
Economia	—	—	—	—	R\$ 64.800,00

Resultado: o threshold ótimo é **0.33**, com custo total de **R\$ 130.400,00** contra **R\$ 195.200,00** no threshold padrão — uma economia de **R\$ 64.800,00** (33.2%) sobre o conjunto de teste de 6.000 leituras. O ajuste desloca o modelo para um regime mais sensível: aceita mais alarmes falsos (baratos) para eliminar falhas não detectadas (caras).

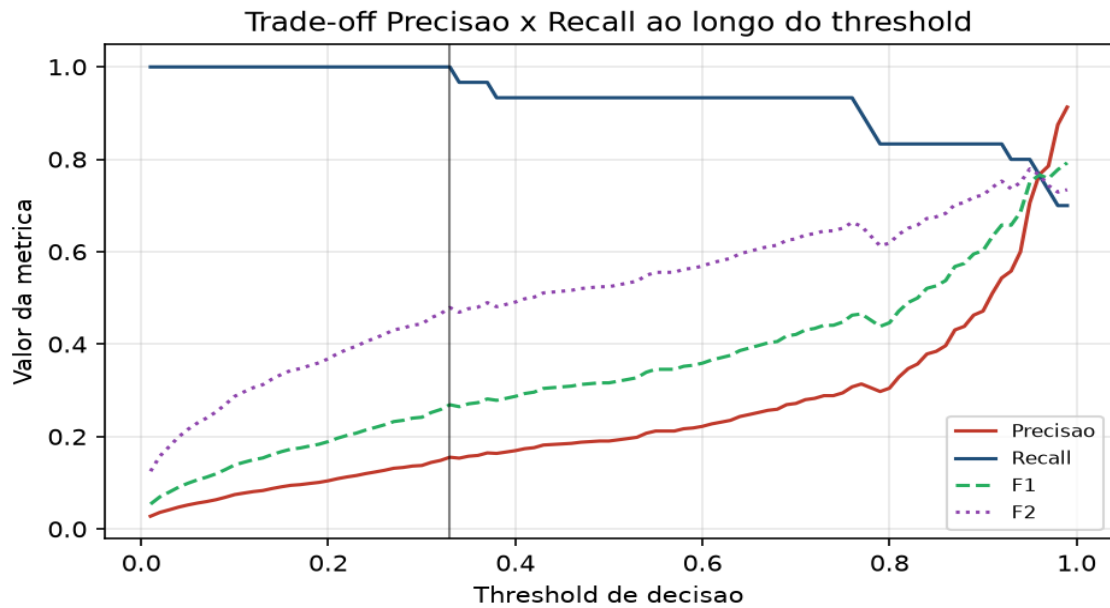


Figura 4 — Trade-off entre precisão, recall, F1 e F2 ao longo do threshold.

4. Recomendações MLOps para Produção

4.1 Monitoramento

- **Métricas de negócio, não acurácia:** o painel de produção deve acompanhar Recall, Precisão, F2 e o custo acumulado estimado ($FN \times \text{custo_FN} + FP \times \text{custo_FP}$). Acurácia não deve aparecer como indicador principal.
- **Data drift:** comparar semanalmente a distribuição de cada sensor em produção com a do treino (teste de Kolmogorov-Smirnov ou PSI). Desgaste mecânico e troca de matéria-prima deslocam as leituras.
- **Concept drift:** após a manutenção confirmar (ou não) cada alerta, recalcular Recall e Precisão em janela móvel de 30 dias e disparar alerta se o Recall cair abaixo de um limiar contratado.
- **Monitoramento da taxa de alertas:** se o volume de predições positivas sair muito da faixa histórica, é sinal de problema no pipeline de ingestão antes mesmo de haver rótulo disponível.

4.2 Governança e reprodutibilidade

- **Versionamento:** código em Git, dados e artefatos (modelo + scaler) versionados juntos, com a semente aleatória registrada. O scaler deve ser serializado com o modelo — usar um scaler diferente em produção é outra forma de leakage.
- **Threshold como parâmetro de configuração:** o valor ótimo depende dos custos de FN e FP, que mudam com o contrato de manutenção. Deve ficar em arquivo de configuração e ser recalculado periodicamente, nunca codificado no meio do script.
- **Retreino:** agendar retreino periódico (mensal) ou disparado por gatilho de drift, sempre repetindo o isolamento do conjunto de teste antes de qualquer transformação.
- **Validação pré-deploy:** nenhum modelo novo sobe sem superar o modelo em produção no conjunto de teste congelado, avaliado por custo total e não por acurácia.

5. Conclusão

Em um cenário com 0.5% de eventos raros, a acurácia é uma armadilha: um modelo que nunca detecta falha alcança 99.50% e ainda assim é inútil. A avaliação correta exige a matriz de confusão como base, métricas derivadas coerentes com o custo do erro (F2 e Recall neste caso), a AUC-ROC para medir a capacidade de ordenação independente do corte e, sobretudo, o ajuste do threshold segundo a realidade financeira da operação — o que gerou uma economia estimada de R\$ 64.800,00 apenas no conjunto de teste.